

Remediation brief — `rewrite/python` (for the implementing AI)

To: the assistant that generated the `rewrite/python` branch. **From:** an independent static audit against `main` (the production PHP bot, which is the source of truth). **Purpose:** a precise, ordered work order to reach real parity. Every item below was found by *reading the code*, not running it. Fix in the order given — the CRITICAL block breaks the revenue path and must land first, together with tests.

Ground rules before you start

1. **Do not trust** `PARITY.md`. Several of its  entries are false (called out below). Re-verify each claim against the code.
 2. **Every fix needs a test that would have failed before it.** The current suite is green *because it never exercises the settlement/checkout path* — that is how the CRITICAL bugs shipped. Add end-to-end coverage for the purchase→pay→provision→notify journey per gateway.
 3. **Do not add features while fixing bugs.** Keep additions labelled and separate, as you already did.
 4. File references are `path::symbol` with line hints from the audited commit; re-locate by symbol if lines drift.
-

CRITICAL — online payments are non-functional (fix first, as one tranche)

C1 — `PaymentService.settle_wallet_topup` is called but never defined

- **Where:** called at `src/mirza/api/payhooks.py:76` and `:107`, `src/mirza/jobs/builtin.py:435` (`_settle`), `src/mirza/payments/stars.py:160`. **No definition exists** anywhere (`PaymentService` in `src/mirza/payments/service.py` only defines `settle_direct_buy`).
- **Effect:** every wallet top-up settlement — Zarinpal, Aqayepardakht, IranPay, Plisio, NowPayments, card-to-card, **and Telegram Stars** — raises `AttributeError` at runtime.
- **Fix:** implement `async def settle_wallet_topup(self, order, cashback_pct: int = 0)` on `PaymentService`: credit `order.price` (+ optional cashback) to `order.user_id` via `WalletService.change(..., reason="topup")`, guard against double-credit (the `claim_paid` gate already covers idempotency, but assert it), and return a result the callers can use.
- **Accept when:** a test tops up a wallet through at least one gateway callback and asserts the balance increased exactly once (and once more under a duplicate callback = no double credit).

C2 — Gateway *product* purchases never provision the product

- **Where:** `src/mirza/handlers/user/buy.py::pay_gateway` calls `PaymentService.create_order(str(db_user.id), int(price), gw_name, bot_type="main")` with **no** `invoice_id`. `payhooks.py::settle` only provisions when `order.invoice_id` is set (`if order.invoice_id: settle_direct_buy(...) else: settle_wallet_topup(...)`).

- **Effect:** a customer who selects a product and pays via an online gateway gets a wallet top-up (which then crashes per C1), **not the config they bought**. The product `code` is discarded (used only in the description string).
- **Fix:** mirror what the Mini-App already does correctly (`src/mirza/api/miniapp.py::mini_purchase`): create the `Invoice` row (status `pending`) *before* the order, and pass `invoice_id=<that token>` into `create_order` . Then `settle_direct_buy` will fire on callback.
- **Accept when:** paying a product order via a gateway callback creates/activates exactly one config for the buyer and no phantom wallet credit.

C3 — Buyer is never notified and the admin report is a no-op

- **Where:** `src/mirza/api/payhooks.py` — `settle / settle_tuple` call `_notify_user(None, order.user_id, text)` (bot hard-coded to `None` , so `_notify_user` returns immediately). `_report()` (`payhooks.py:41`) builds `chat` then does `_ = chat, kind, text # delivery via scheduler ctx in production run` and sends nothing.
- **Effect:** successful payments are silent to the customer, and no payment report reaches the admin channel/topic (a rich feature in `main`).
- **Fix:** thread a real `Bot` instance into the settlement path (via the aiohttp app context / job context) and actually send the buyer DM; implement `_report()` to post to the configured report channel/topic (resolve `topicid` for `paymentreport`). Do not swallow the send silently — log failures.
- **Accept when:** a settled payment produces a buyer DM and an admin-channel message in an integration test with a fake bot.

C4 — Wallet purchase provisions and commits *before* charging

- **Where:** `src/mirza/handlers/user/buy.py::pay_wallet` . Order of operations: `panels.create_user(...)` (which **commits** the invoice inside `panels/service.py::create_user` , line ~126) → then `WalletService.change(-price)` . On failure it calls `session.rollback()` , but the invoice is already committed and the config already exists on the remote panel.
- **Effect:** a failed/insufficient charge yields a **free provisioned config**. Because balance is checked earlier in `buy_confirm` and deducted later, two concurrent purchases can both pass the check and both provision → double-spend.
- **Fix:** charge the wallet atomically *first* (conditional decrement: `UPDATE user SET balance = balance - :price WHERE id=:id AND balance >= :price` returning affected rows), and only provision if the charge succeeded; on provision failure, refund in the same transaction. Never commit the invoice before the charge is secured.
- **Accept when:** a concurrency test with balance = one purchase's price and two simultaneous buys results in exactly one config + one debit.

● HIGH — regressions and broken behaviour

H1 — `change_location` creates a dead, empty config

- **Where:** `src/mirza/panels/service.py::change_location` → `create_user(new_panel, product_code, username, data_limit=0, expire_ts=_now())` .

- **Effect:** the migrated config has **zero volume and expires immediately**; the user's remaining quota/time is dropped. `main`'s `changeloc` preserves remaining volume and time.
- **Fix:** read the current config from the old panel (`data_user`), compute remaining volume (`data_limit - used_traffic`) and remaining time (`expire_at - now`), create on the new panel with those, then remove from the old.

H2 — `manual_sell` provisions zero volume and ignores the product

- **Where:** `src/mirza/handlers/admin/__init__.py::manual_sell` → `create_user(..., data_limit=0, expire_ts=now+30d)`, and `_default_panel_name` picks the first panel arbitrarily.
- **Fix:** look up the product by `code`, use its `volume_gb` / `service_days` / `location`; fail clearly if the product doesn't exist.

H3 — Expiry & volume reminders go to the admin channel, not the customer

- **Where:** `src/mirza/jobs/builtin.py` — `NotificationsJob` and `VolumeWarnJob` only call `ctx.send_report("notifications", ...)`. `main`'s `NoticationsService.php` DMs the *customer* with renew buttons.
- **Effect:** the renewal-reminder engine never reaches buyers — a direct revenue regression.
- **Fix:** DM `inv.user_id` with a renew CTA; optionally also mirror to the report channel. Respect the per-invoice `notifications` flags (already present) to avoid repeats.

H4 — `disableconfig` never persists its status change

- **Where:** `src/mirza/jobs/builtin.py::DisableConfigJob` disables on the panel and sets `inv.status = "disabled"` but has **no** `await s.commit()` (unlike sibling jobs).
- **Effect:** status change is rolled back on session close; the job re-disables the same invoices every tick and the DB never reflects reality.
- **Fix:** commit at the end of the sweep.

H5 — Multiple admins unimplemented (contradicts `PARITY.md` "✅")

- **Where:** `src/mirza/handlers/admin/__init__.py::AdminGate` ("only primary admin passes") and `manage.py::AdminGate` gate solely on `settings.admin_number`. The `Admin` table and `rule` column are never consulted for bot gating.
- **Fix:** gate on membership in the `admin` table (with `rule` /level checks where `main` differentiates), not a single env id. Keep the primary admin as a superuser.

H6 — `uptime_node` authenticates with the *encrypted* password

- **Where:** `src/mirza/jobs/builtin.py::UptimeNodeJob` builds `MarzbanAdapter({... "password_panel": panel.password_panel_encrypted})` — the Fernet ciphertext, not the plaintext.
- **Fix:** decrypt via the same path `PanelService._adapter_for` uses (`_decrypt_secret`), or route through `PanelService` instead of constructing the adapter directly.

H7 — Migrations are `create_all` in disguise, with dead code

- **Where:** `alembic/versions/0001_initial.py::upgrade` loops `Base.metadata.sorted_tables` and calls `table.create()` ; lines ~30–42 build `col_defs` via `col.compile(...)` and **never use them**. No real DDL is captured, and the compatibility view the `Setting` model docstring promises (for the legacy flat `setting` columns) does not exist.
- **Fix:** generate a real autogenerated migration with explicit `op.create_table(...)` per table; add the `setting` compatibility view (or a data migration) if you keep the KV redesign; delete the dead `col_defs` loop.

H8 — Process-local state breaks restarts and multi-worker

- **Where:** module-level dicts: `buy._pending` , `misc._TOPUP` , `admin.__init__._BROADCAST` , `routes/webpanel._SESSIONS` .
- **Effect:** all in-flight purchase/top-up/broadcast/admin-session state is lost on restart and is not shared across workers; also enables the C4 race.
- **Fix:** use aiogram FSM with a persistent (Redis) storage backend, and persist web sessions in the DB or a shared store. `main` persisted step state in the DB (`user.step` , `Processing_value*`).

🟡 MEDIUM — incomplete / disconnected features

M1 — Discount engine ignores most of the model

- **Where:** `src/mirza/handlers/user/buy.py::discount_code` applies only `price_discount` (fixed). `discount_percent` is ignored, `used_count` is never incremented (so `usage_limit` is unenforceable), `expires_at` is unchecked, and `DiscountSell` (per-product codes) is unused.
- **Fix:** apply percent *and* fixed, enforce `usage_limit` by incrementing `used_count` transactionally at redemption, honour `expires_at` , and support `DiscountSell` .

M2 — Gift codes are inert and the consumed-count is wrong

- **Where:** `manage.py::add_giftcode` creates a `Discount` with `discount_percent=0` / `price_discount=0` → no actual discount. `manage.py::list_giftcodes` builds `{g.code: g for g in ...}` (keyed by code), so `used` collapses to ≤ 1 entry per code and the consumed count is always 0/1.
- **Fix:** let `/giftcode` set a real value; count consumption with `COUNT(*) GROUP BY code` over `Giftcodeconsumed` .

M3 — Lucky-wheel and wheel-prize config are disconnected

- **Where:** `misc.py::lucky_wheel` uses a hard-coded `WHEEL_PRIZES` ; `manage.py::set_wheel_prizes` writes the `wheel_prizes` setting that the wheel never reads. Legacy wheels also support weighted probabilities.
- **Fix:** read `wheel_prizes` from settings; support weights if `main` does.

M4 — Scheduled lottery missing; gift job stubbed

- **Where:** `main` has `cronbot/lottery.php` (periodic draw) — **no equivalent job** in `jobs/builtin.py` (header even says "16 jobs"; `main` has 17). `GiftJob.run` is `return None`.
- **Fix:** port the lottery draw; implement scheduled gift drops per `gift.php`.

M5 — Referral commission & cashback not implemented

- **Where:** `menu.py` captures `referrer_id` and `misc.py::affiliates` shows a count, but nothing pays commission on a referred purchase. `Affiliate.porsant_one_buy`, `reagent_report`, `status_commission` are unused; the only "cashback" is a hard-coded `cashback = 0` in `jobs/builtin.py::_settle`.
- **Fix:** on a referred user's first/every purchase (per `main`'s rule), credit the referrer's wallet and record `reagent_report`; implement cashback where `main` grants it.

M6 — Free trial ignores per-panel config; weak gating

- **Where:** `misc.py::free_trial` hard-codes 1 GB / 3 days and passes `product_code=""`; it ignores the panel's `time_usertest` / `val_usertest`, and gates only on "already has a test invoice" (no phone/channel/limit checks; `on_hold_test` unused).
- **Fix:** read per-panel trial volume/time; apply the same gates `main` enforces.

M7 — On-hold path is dead; `service_time` semantics are inconsistent

- **Where:** `OnHoldJob` processes `status == "on_hold"`, but **no handler ever sets `on_hold`**. Also `create_user` writes an **absolute expiry timestamp** into `invoice.service_time`, while `OnHoldJob` (`_days_s`) and `service._expire_from` treat `service_time` as a **day count** → nonsensical expiry math.
- **Fix:** pick one representation for `service_time` and use it everywhere; wire the on-hold creation path so the job has something to activate.

M8 — Forced-channel join is single-channel and fails open

- **Where:** `middleware.py` enforces only a single `channel_lock` setting; the multi-row `channels` table (populated by `/addchannel`) is never enforced. `_check_member` does `channel.lstrip("@")` (likely mis-resolves the chat) and returns `True` on exception (fail-open).
- **Fix:** enforce all rows in `channels`; pass a valid chat identifier to `get_chat_member`; decide fail-open vs fail-closed deliberately (legacy fail-open is defensible but note it).

M9 — Admin surface is ~25 commands vs ~177 in `main`

- Missing/critically-thin admin capability (implement to reach parity): **in-bot text/message customization** (edit any bot string — a headline `main` feature, entirely absent); **per-panel pricing/inbound/test-account/`method_username`/extend editing**; **product edit/delete**; **card-number management**; **report-topic configuration**; user limits (`maxbuyagent` / `changeloclimit` / `expireset` / verify); broadcast with media/pin/forward (currently text only, and it opens one DB session per user — batch it); agent types/pricing (`n` / `n2`). Also add multiple username-generation methods (only `random` / `number` exist).

M10 — Mini-App wallet purchase never completes

- **Where:** `api/miniapp.py::mini_purchase` with `method="wallet"` creates a pending order but nothing deducts the wallet or settles it (no callback for the wallet method).
 - **Fix:** settle wallet-method purchases inline (charge + `settle_direct_buy`) instead of leaving a dangling pending order.
-

● LOW — hygiene, dead code, config safety

- **L1 — Remove dead/placeholder code (AI-gen tells):** the shadowed **insecure** duplicate `miniapp` in `api/__init__.py` (lines ~73-124; its `token` action trusts a client `user_id` — dangerous if ever re-wired); `payhooks._report` no-op; `service._expire_from` (unused); the migration `col_defs` loop; `api/__init__.py:320` `r = web.Application() # placeholder to keep linters calm`; `admin.__init__._TOPUP_UNUSED` ; `manage.new_tracking_code` .
 - **L2 — `protocol` table missing** from the model (present in `main`). Add it if protocol-level config is used.
 - **L3 — Web panel is list-only** (`routes/webpanel.py`): no product/panel/user/discount editing. Either build the CRUD or make the bot the source of truth — but something must be able to edit these.
 - **L4 — Config safety:** `MIRZA_SESSION_SECRET` default is `change-me` and it doubles as the Fernet key for panel passwords → creds encrypted under a known key. Refuse to boot on the default; use a **separate** key for panel encryption vs. web sessions. The webhook-secret check and channel check both fail open when unset — require them explicitly.
 - **L5 — Web panel security nits:** set the session cookie `Secure` ; escape all columns in `_table` (currently inconsistent); the bcrypt timing-parity "dummy hash" is not a valid bcrypt string and raises (defeating the intent). SQL is parameterized via the ORM — no injection found, keep it that way.
-

Suggested sequencing

1. **C1-C4 together, with an end-to-end money-path test** (this is the whole point of the product).
 2. **H1-H8** (regressions + infra).
 3. **M1-M10** (feature completeness).
 4. **L1-L5** (hygiene) — L1 and L4 are quick and worth doing alongside C/H.
-

What was NOT statically checkable (verify while you fix)

Panel adapters against real panels (only `marzban.py` was read in depth — it looks faithful; Marzneshin, Alireza/x-ui, S-UI, Hiddify, WGDashboard keygen, MikroTik REST v7, IBSng, Pasarguard, Rebecca, mirza_agent need live round-trips); Alembic up/down on real PostgreSQL; each gateway's verify logic against live sandboxes; the installer; white-label child-bot dispatch.